Technical Report: StockPilot - An AI Financial Market Research Agent

1. Introduction

StockPilot is an AI-powered financial market research agent designed to automate the first stage of stock analysis. The goal of the project is to reduce the amount of time users spend gathering financial information from separate sources such as stock market platforms, price charts, company profiles, and financial news websites. The system allows a user to enter a stock ticker and automatically generates a structured financial research report.

The primary users of StockPilot include individual investors, finance students, business students, and analysts performing preliminary company research. The project was designed as an agentic AI product that combines financial data retrieval, visualization, news extraction, and large language model summarization into a single workflow.

2. Problem Statement

Financial research is often fragmented. A user researching a company may need to check stock price history, company information, recent news headlines, market performance, and then synthesize the results into a clear summary. This process can be time-consuming, especially for beginner investors or students.

StockPilot addresses this issue by creating a streamlined research pipeline. Instead of manually searching across multiple platforms, the user enters a stock ticker such as AAPL, NVDA, or TSLA, and the system retrieves relevant data, calculates recent price performance, displays a price chart, collects recent headlines, and uses Gemini to generate an AI-written financial research report.

3. System Overview

StockPilot is implemented as a lightweight web application using Streamlit. The application integrates multiple tools:

- **Streamlit** for the user interface
- **yfinance** for stock data and company information
- **pandas** for data handling
- **Plotly** for interactive price visualization
- **Google Gemini API** for AI-generated report writing

- **python-dotenv** for secure API key loading

The system follows a sequential agentic workflow:

1. User enters a stock ticker.
2. The application retrieves financial data from Yahoo Finance using `yfinance`.
3. The application extracts company metadata and six months of historical price data.
4. The system calculates the six-month percentage change.
5. The application displays key metrics and an interactive price chart.
6. The system retrieves recent news headlines.
7. A structured prompt is sent to Gemini.
8. Gemini generates a financial research report.
9. The report is displayed in the Streamlit interface.

4. Agent Framework and Workflow

Although StockPilot does not use a complex multi-agent framework such as LangGraph, it still follows an agentic design because the application performs multiple coordinated actions before producing the final output.

The agent workflow can be described as:

Input → Data Retrieval → Data Processing → Visualization → Prompt Construction → LLM Report Generation → Output

The user only provides one input: the stock ticker. After that, the application automatically decides what information to retrieve and how to structure it for the final report.

5. Data Retrieval Pipeline

The financial data pipeline begins with:

```
stock = yf.Ticker(ticker)
hist = stock.history(period="6mo")
```

The application retrieves six months of historical stock price data. If no data is found, the system stops and displays an error message. This prevents the application from generating inaccurate or empty reports for invalid tickers.

The app also retrieves company-level metadata using:

```
info = stock.info
```

From this dictionary, the application extracts:

- Company name
- Sector
- Industry
- Market capitalization

These values are then used both in the Streamlit dashboard and in the Gemini prompt.

6. Financial Calculations

StockPilot calculates the six-month price change using the first and last closing prices from the historical data:

```
current_price = hist["Close"].iloc[-1]
start_price = hist["Close"].iloc[0]
percent_change = ((current_price - start_price) / start_price) * 100
```

This provides a simple but useful measure of recent stock performance. The six-month change helps users quickly understand whether the stock has increased or decreased over the selected time period.

7. User Interface

The user interface is built using Streamlit. The app uses a wide layout and displays the title:

StockPilot: AI Financial Market Research Agent

The user enters a stock ticker into a text input box. When the user clicks **Generate Report**, the app runs the full pipeline.

The dashboard displays four key metrics:

- Company
- Current Price
- Six-Month Change
- Sector

The interface is designed to be simple and beginner-friendly, making it useful for finance students or users who are new to market research.

8. Data Visualization

StockPilot uses Plotly to create an interactive six-month closing price chart. The chart displays the company's closing price over time, allowing users to visually inspect recent price movement.

The chart is created using:

```
fig = go.Figure()
fig.add_trace(
  go.Scatter(
     x=hist.index,
     y=hist["Close"],
     mode="lines",
     name=f"{ticker} Close Price"
  )
)
```

This visualization helps users understand the stock's recent trend before reading the AI-generated commentary.

9. News Retrieval

The application attempts to retrieve recent news items using:

```
news_items = stock.news[:5]
```

The system extracts up to five recent headlines and formats them as bullet points. If no headlines are available or if the news retrieval fails, the application uses the fallback message:

```
No recent headlines available.
```

This makes the system more reliable because the report can still be generated even if recent news data is unavailable.

10. LLM Integration

StockPilot uses Google Gemini through the `google-genai` package. The Gemini API key is loaded from the environment using:

```
api_key = os.getenv("GEMINI_API_KEY")
```

If the API key is missing, the app displays an error and stops. This ensures that the user understands what must be configured before running the product.

The system then creates a structured prompt containing:

- Company name
- Ticker
- Sector
- Industry

- Market cap
- Current price
- Six-month price change
- Recent news headlines

The prompt instructs Gemini to generate a report with the following sections:

1. Company Overview
2. Recent Price Trend Summary
3. News Highlights
4. AI Market Commentary
5. Key Risks or Limitations
6. Disclaimer that this is not financial advice

The model used is:

gemini-2.5-flash

This model is appropriate for the project because it can quickly generate structured explanations from retrieved financial data.

11. Implementation Details

The project is implemented in a single Python file. This makes the application simple to run and easy to evaluate. The app uses `dotenv` to load the Gemini API key locally, which avoids hardcoding sensitive credentials directly in the code.

The application also includes several reliability checks:

- Stops if the API key is missing
- Stops if the ticker field is empty
- Stops if no stock data is found
- Uses fallback behavior if news headlines cannot be retrieved

These checks improve the robustness of the product and make the user experience clearer.

12. How to Run the Product

To run StockPilot, the user should first install the required packages:

pip install streamlit yfinance pandas plotly google-genai python-dotenv

Then edit a `.env` file in the project folder with:

GEMINI_API_KEY=your_api_key_here

After that, run the app using:

streamlit run app.py

The application will open in a browser. The user can enter a stock ticker and click **Generate Report** to produce the dashboard and AI financial research report.

13. Limitations

StockPilot is designed for preliminary financial research, not professional investment advice. The system has several limitations:

First, the application depends on Yahoo Finance data through `yfinance`, which may occasionally have missing or delayed information.

Second, the news retrieval function depends on the availability of headlines through the stock object. If the news data is unavailable, the app cannot provide current news analysis.

Third, the AI-generated report is based only on the information included in the prompt. It does not independently verify all financial claims beyond the retrieved data.

Fourth, the current version does not include advanced financial ratios, earnings analysis, balance sheet data, cash flow analysis, or analyst forecasts.

Finally, the output should not be interpreted as financial advice. The report is intended for educational and research support only.

14. Future Improvements

Future versions of StockPilot could include:

- More detailed financial ratios
- Earnings report summaries
- Analyst recommendation data
- SEC filing analysis
- Sentiment analysis of news headlines
- Multiple-stock comparison
- Portfolio-level analysis
- Downloadable PDF reports
- A more advanced agent framework such as LangGraph
- Retrieval-augmented generation using financial documents

These improvements would make the system more powerful and closer to a full financial research assistant.

15. Conclusion

StockPilot successfully demonstrates an AI-powered agentic workflow for financial market research. The system takes a simple stock ticker as input, retrieves financial data, calculates recent performance, visualizes price history, collects recent headlines, and generates a structured AI research report using Gemini.

The project combines API integration, data processing, visualization, prompt engineering, and LLM-based report generation into a single runnable product. Overall, StockPilot shows how agentic AI systems can simplify fragmented research tasks and help users quickly understand a company’s recent market activity.